

# Lab8 Sparse Conditional Constant Propagation

---

## 1. QuadProgram\* optProg(QuadProgram\* prog)

---

遍历所有函数，初始化执行优化所需的Opt类，包括当前优化的函数指针、每个块是否可达的map、对temp分析得到的类型的map。此外还包括一个新建的 `map<Opt*, set<pair<int, int>>>` `executable_edges` 用于记录哪条边是可达的而不仅仅是块，这样便于后续处理 Phi 函数。

## 2. QuadFuncDecl\* Opt::optFunc()

---

按顺序执行如下两个函数：

### 2.1 calculateBT()

B = Block executable，计算每个基本块是否可能被执行。

T = Temp value，计算每个临时变量在运行时的值的状态。

函数的参数初始化为 `MANY_VALUES`，来自调用者，无法确认。

然后扫描所有基本块，初始化 `label2block`。用 `map<int, int> temp_def_label` 记录每个temp在哪个块定义：对于 MOVE/LOAD/MOVE\_BINOP/MOVE\_CALL/MOVE\_EXTCALL/PHI/PTR\_CALC 语句，会有新的 SSA temp 被定义。

预处理结束后，将函数的入口块标注为可达，然后进行一个循环迭代计算块的可达性，直到 `block_executable` 以及 `tempValue` 不发生变化。

#### 2.1.1 MOVE

将右值的 `termValue` 给左值。

#### 2.1.2 MOVE\_BINOP

分析左右操作数，若均为常量，尝试进行求值。除以0的操作不进行求值，保留 `MANY_VALUES`。

若一个不为常量，则结果也不是常量。

#### 2.1.3 LOAD

内存值当然不是常量，设为 `MANY_VALUES`。

#### 2.1.4 STORE

递归处理 `src` 和 `dst`。

#### 2.1.5 CALL/EXTCALL

递归处理参数。

## 2.1.6 MOVE\_CALL/MOVE\_EXTCALL

函数返回值认为无法静态确定，设为 `MANY_VALUES`。

## 2.1.7 PTR\_CALC

指针结果认为不是常量，设为 `MANY_VALUES`。

## 2.1.8 RETURN

递归处理 `exp`。

## 2.1.9 JUMP

直接跳转。因为当前分析的块均为可达块，所以传递，指向的块也设为可达块，并且跳转的边设为可达边。

## 2.1.10 CJUMP

先取左右操作数进行分析。然后若均为常量，则直接进行条件判断，得到哪条边可达哪条边不可达，进行相应处理。若有一个为 `MANY_VALUES`，则条件不可确定，两条边均标记为可达。

若有一个 `temp` 当前为 `NO_VALUE`，说明情况未知，先不进行处理。当后续迭代过程中 `temp` 的情况被分析出来后再处理这条 `CJUMP`。

## 2.1.11 PHI

对于 `Phi` 函数的每个参数，检查它记录的块到当前块的边是否可达，如果不可达，跳过这个参数。对于可达的参数，看它的 `temp` 的情况并叠加到 `Phi` 函数的结果。如果某个 `temp` 为 `NO_VALUE`，检查它的定义块是否可达。如果不可达，则这个临时变量是未知的。根据实验说明：

如果某个临时变量在可达路径上被使用时仍然没有确定值，那么这说明程序中出现了未定义值的使用。这种情况应当报告错误，但优化过程仍应继续，并将该值提升为 `MANY_VALUES` 后再继续分析。

我将其提升为 `MANY_VALUES` 并继续分析。

## 2.2 modifyFunc()

主要功能包括：

1. 删除不可达基本块
2. 删除可以被常量替换的赋值
3. 把已知常量替换到使用处
4. 简化 `CJUMP`
5. 修正 `Phi` 参数
6. 必要时给 `Phi` 的常量输入生成新的临时变量

首先，遍历可达块里的 `Phi`，如果 `Phi` 函数的结果为 `ONE_VALUE`，则这个 `Phi` 函数不再需要，后续使用这个 `temp` 的地方替换为对应的常量。

丢掉不可达前驱的参数。若 `Phi` 函数只有一个有效输入，则从 `Phi` 函数改为 `MOVE` 语句。

对于 `Phi` 函数输入 `temp` 为常量的情况，本来已知常量应该被替换到使用处，但是由于 `PHI` 的结构设置中，参数只能是 `pair<Temp*, Label*>`，所以这里不能替换。但是原来的那个 `temp` 作为常量，在其他使用的地方会被替换掉，原来的 `temp` 会被删除。因此此处新建一个 `temp`，专门供给 `Phi` 函数使用，也避免了其他地方替换常量出现问题。并且也需要在前驱块的末尾插入一个针对这个 `temp` 的

MOVE 语句。

根据上面的逻辑，将每个 Phi 函数的处理方法预存储在一个 map 中，随后进行真正的语句修改，构建新的 `QuadFuncDecl`。

参数保持不变，复制。构建新的块列表，移除不可达块。对可达的块创建新的语句列表，对语句重写，删除无用的语句，以及替换常量。

在过程中，重建 def/use。

### 2.2.1 MOVE/MOVE\_BINOP

如果 dst 是常量，并且它不是供给 Phi 函数使用的 temp，则这个 MOVE 语句无作用，删除。

### 2.2.2 LOAD

exp 中的 temp，可以替换为常量。

### 2.2.3 STORE

src 和 addr 中的 temp 可以替换常量。

### 2.2.4 CALL/EXTCALL

替换参数中的常量。

### 2.2.5 MOVE\_CALL/MOVE\_EXTCALL

替换参数中的常量。

### 2.2.6 JUMP

调用 `phi_const_moves`，因为离开块都是在 JUMP 离开。因此如果当前 block 是某个保留 Phi 的前驱，并且需要为 Phi 常量输入生成新的 MOVE，那就在跳转离开当前 block 前插入这些 MOVE。

### 2.2.7 CJUMP

如果条件已知，则改为 JUMP。并且，同样进行和 JUMP 相同的 MOVE 插入。条件判断中的操作数中的常量需要进行替换。

### 2.2.8 PHI

根据预处理中得到的改写方式进行改写/删除/保留（参数修正）。

### 2.2.9 ExitLabels

只保留可达边的出口块。